Laboratory Session 3

Target tracking for a Turtlebot robot using Model Predictive Control

Ionela Prodan

2025

Contents

1 Optimization-based control **2**
1.1 Model Predictive Control - Background 2
1.2 MPC implementations 3
1.3 Target tracking of a Turtlebot robot using MPC 4

2 Exercises **5**

Throughout this session we will introduce some basic definitions in optimization-based control and we will concentrate on MPC (Model Predictive Control) implementations using CasADi in Matlab, an open-source modeling language for nonlinear optimization.

We consider the example of target tracking with a TurtleBot robot.

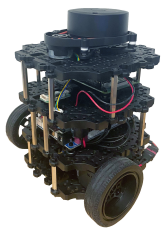

Figure 1: Turtlebot robot.

1 Optimization-based control

Optimization-based control in general terms refers to the control design using an optimization criterion and the respective resolution techniques in order to obtain the parameters of the control law, the optimality being generally equivalent to a certain desired property as for example, stability, reactivity or robustness.

A widely used optimization-based control technique in this class is Model Predictive Control (MPC)[1] also called, *receding horizon control* [2].

1.1 Model Predictive Control - Background

The idea behind MPC is to exploit in a receding manner the simplicity of the open-loop optimization-based control [5, 1, 6]. The control action $u(k)$ for a given state $x(k)$ is obtained from the control sequence $\mathbf{u} \triangleq \{u(k), u(k+1), \ldots, u(k+N_p-1)\}$ as the result of the optimization problem [4]:

$$\arg\min_{\mathbf{u}} \quad V_f(x(k+N_p)) + \sum_{s=0}^{N_p-1} V_n(x(k+s), u(k+s)), \tag{1}$$

$$\text{subject to:} \quad \begin{cases} x(k+s+1) = f(x(k+s), u(k+s)), \quad s = 0, \ldots, N_p-1, \\ h(x(k+s), u(k+s)) \leq 0, \quad s = 0, \ldots, N_p-1, \\ h_f(x(k+N_p)) \leq 0, \end{cases} \tag{2}$$

over a finite horizon N_p. The cost function is comprised of two basic ingredients; namely a terminal cost function $V_f(\cdot) : \mathbb{R}^n \to \mathbb{R}$ and a cost per stage function $V_n(\cdot) : \mathbb{R}^n \times \mathbb{R}^m \to \mathbb{R}$. Typically, in MPC, the objective (or cost) function (1) penalizes deviations of the states and inputs from their reference values, while the constraints are treated *explicitly* [3]. By solving (1), each optimization generates an open-loop optimal control trajectory taking into account the system dynamics described by $f(\cdot,\cdot)$, generally with the additional property that f(0,0)=0, the constraints on the states and control inputs $h(\cdot)$ and terminal constraint $h_f(\cdot)$. Then, applying only the first part of the trajectory to the system, based on measurement and recomputing, results in closed-loop control (see, Fig. 1 which illustrates very well the receding horizon strategy).

The restrictions (2) of the optimization problem (1) can be written in a more explicit form, stated in terms of hard constraints on the internal state variables and input control action (whenever these are separable):

$$\begin{cases} x(k+s+1) = f(x(k+s), u(k+s)), \quad s = 0, \ldots, N_p-1, \\ x(k+s) \in \mathcal{X}, \quad s = 0, \ldots, N_p-1, \\ u(k+s) \in \mathcal{U}, \quad s = 0, \ldots, N_p-1, \end{cases} \tag{3}$$

[1] The terminology "Model Predictive" comes from the use of the model to predict the system behavior over the planning horizon at each update.

[2] The terminology "receding horizon" comes from the fact that the planning horizon, which is typically fixed, moves ahead in time with each update.

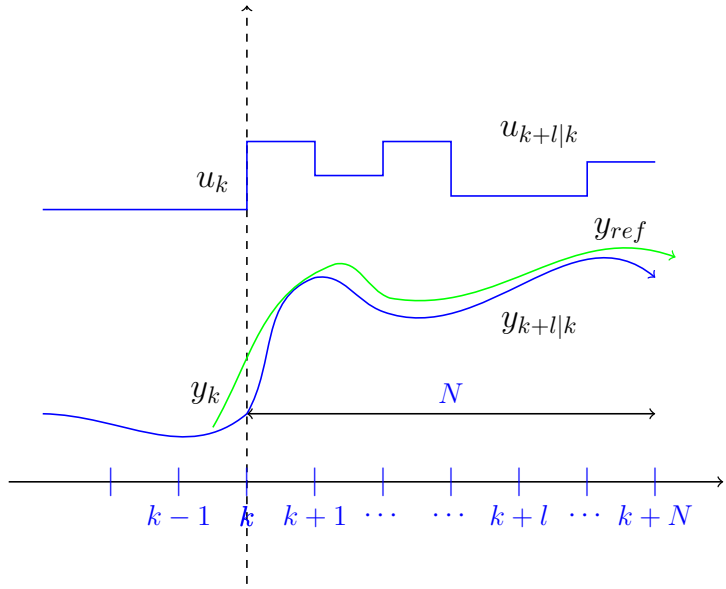

Figure 2: Receding horizon control philosophy.

where, usually, $\mathcal{X}$ is a convex, closed subset of $\mathbb{R}^n$ and $\mathcal{U}$ is a convex, compact subset of $\mathbb{R}^m$, each set containing the equilibrium point in their strict interior (generally $f(0,0) = 0$ and $0 \in \mathcal{X}$, $0 \in \mathcal{U}$). A terminal constraint could also be imposed for stability reasons mayne2009model:

$$x(k + N_p) \in \mathcal{X}_f \subset \mathcal{X}. \tag{4}$$

MPC represents one of the few methods in control that can handle generic state and control constraints. More precisely, it has the ability to include generic models (i.e., nonlinear and linear) and constraints in the optimization-based control problem (1)–(2). In addition, to this main advantage, it is worth mentioning its' capacity to redefine the cost function and the constraints to account for the changes in the system and/or the environment. In our opinion, the major inconvenient of receding horizon strategy is represented by the computational demand, i.e., it is prohibitive the requirement that an optimization algorithm must run and terminate at every update of the controller block (generally synchronous with the sampling clock).

1.2 MPC implementations

All computations in MPC are based on a discrete time representation of the system dynamics. On state space form the model is:

$$x(k+1) = Ax(k) + Bu(k), \tag{5}$$

$$y(k) = Cx(k), \tag{6}$$

where 1 time step corresponds to the sample time T_e. There is never a direct term D from input to output in MPC models. This follows from the fact that, at a given sample k, the output is already given and can hence not be affected by the present input $u(k)$ which is to be calculated by the optimizer.

The general MPC formulation is provided in (1) where the cost function can be quadratic

$$V_n(x(k+s), u(k+s)) = x^T(k+s)Qx(k+s) + u^T(k+s)Ru(k+s), \tag{7}$$

linear

$$V_n(x(k+s), u(k+s)) = f^T x(k+s) + g^T u(k+s), \tag{8}$$

or nonlinear.

For implementing MPC problems in Matlab there are several options:

- There are a set of Matlab functions, referred to as MPCtools. MPCtools implements an MPC controller for use with Matlab/Simulink. MPCtools requires no installation or compilation, but it is convenient to add the directory containing the MPCtools functions to the Matlab path. The tools require Control System Toolbox and, if the Simulink extension is to be used, also Simulink.
- The quadratic programming (QP) solver *"quadprog"* may be used to solve the MPC optimization problem or the linear programming (LP) solver *"linprog"*. This feature requires Matlab Optimization Toolbox.
- MPC Toolbox 3.0 can also be used.
- Yalmip toolbox [2] permits to write optimization problems (cost and constraints) in a compact manner by hiding the complexities of constraint concatenation (thus saving time and errors).
- as indicated in the first Laboratory session, we will mostly use CasADi modeling language since it allows Octave, Matlab and Python implementation.

1.3 Target tracking of a Turtlebot robot using MPC

The kinematics of the WMR (wheeled mobile robot) can be described as:

$$\begin{aligned} \dot{\xi}_1 &= v\cos\xi_3, \\ \dot{\xi}_2 &= v\sin\xi_3, \\ \dot{\xi}_3 &= \omega, \end{aligned} \tag{9}$$

where $\boldsymbol{\xi} = [\xi_1\ \xi_2\ \xi_3]^\top$ denote the positions and heading angle of the robot, respectively (see Fig. 3). The input of the system is the translational, v (m/s), and rotational, ω (rad/s), velocity. These velocities are constrained by the actuator limits as:

$$|v| \leq \bar{v} \tag{10}$$

$$|\omega| \leq \bar{\omega}. \tag{11}$$

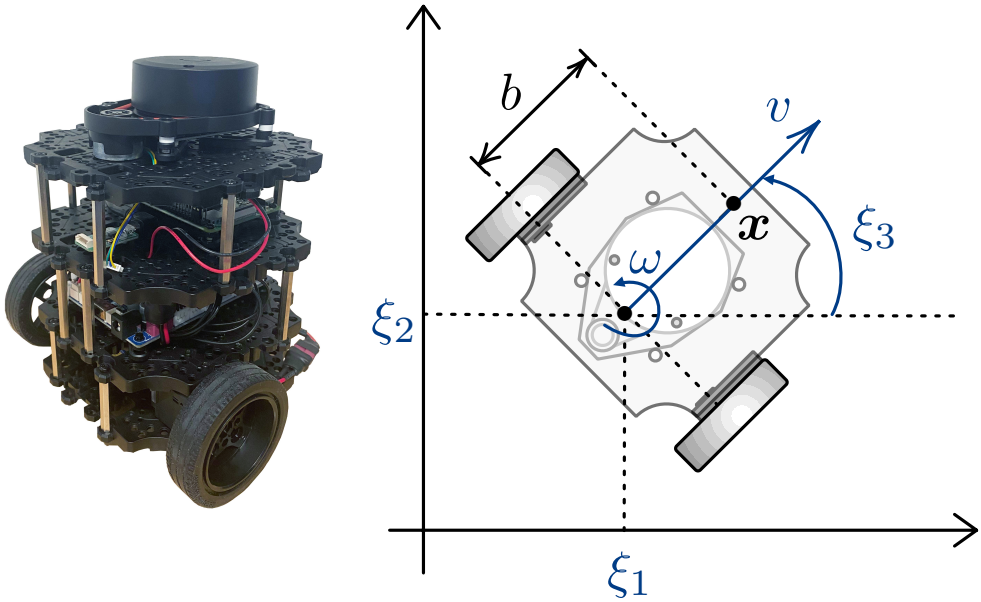

Figure 3: Wheeled mobile robot model

In Figure 3 given $b > 0$, consider the following output vector:

$$\boldsymbol{x} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \Phi(\boldsymbol{\xi}) = \begin{bmatrix} \xi_1 \\ \xi_2 \end{bmatrix} + b \begin{bmatrix} \cos \xi_3 \\ \sin \xi_3 \end{bmatrix} \tag{12}$$

then, by means of the transformation:

$$\begin{bmatrix} v \\ \omega \end{bmatrix} = T_{FL}(\xi_3)\boldsymbol{u} = \begin{bmatrix} \cos \xi_3 & \sin \xi_3 \\ -\frac{\sin \xi_3}{b} & \frac{\cos \xi_3}{b} \end{bmatrix} \boldsymbol{u} \tag{13}$$

the system (9) yields:

$$\dot{\boldsymbol{x}} = \boldsymbol{u}. \tag{14}$$

In this setting, the control problem now becomes the stabilization of a simple single integrator as in (14).

2 Exercises

The next exercises are using the code provided at the following link

Exercise 2.1. Explain how the linear continuous-time system in (14) was obtained.

Exercise 2.2. Obtain the state space representation of the linear continuous-time system in (14).

Exercise 2.3. Discretize the linear continuous-time system in (14) with a sampling time of $T_s = 0.1$. Consider $\dot{\boldsymbol{x}} = (x_{k+1} - x_k)/T_s$.

Exercise 2.4. For N_{pred}, the prediction horizon, Q, R and P weighting matrices of appropriate dimensions, write the target tracking problem or the linear discrete-time system. Obtain the optimal controller and apply it for the nonlinear model of the robot in (9).

To compare this method, write as well the LQR control problem for the same system.

To be able to answer the next questions it will be very useful to provide a table which shows:

- initial conditions
- N_p, Q, R, P (the MPC tuning)
- Computing time

Exercise 2.5. Choose the weighting parameters the same, $Q = R = P$ and consider values of the prediction horizon from 1 to 10. What happens? Explain the observed behavior. Compute the computation time for each prediction horizon.

Exercise 2.6. Change the prediction horizon and increase/decrease the terminal weight. Run a simulation. Comparing with the result above, what is the effect of increasing/decreasing the terminal weight?

Exercise 2.7. Keep the prediction horizon fixed and increase/decrease the cost per stage and input weights. Run a simulation. Comparing with the result above, what is the effect of increasing/decreasing the cost per stage weight?

Exercise 2.8. Change the sampling time. What happens? Try to explain the observed behavior.

Exercise 2.9. Provide comments on the comparison between the MPC and LQR control for this particular robot model.

References

[1] C.R. Cutler et al. *Method for removal of PID dynamics from MPC models.* US Patent 7,263,473. 2007.

[2] Johan Löfberg. "YALMIP: A toolbox for modeling and optimization in MATLAB". In: *Computer Aided Control Systems Design, 2004 IEEE International Symposium on.* IEEE. 2004, pp. 284–289.

[3] David Q. Mayne et al. "Constrained model predictive control: Stability and optimality". In: *Automatica* 36 (2000), pp. 789–814.

[4] R.M. Murray. "Optimization-Based Control". In: *Technical Report, California Institute of Technology, CA* (2009).

[5] A.I. Propoi. "Use of linear programming methods for synthesizing sampled-data automatic systems". In: *Automation and Remote Control* 24.7 (1963), pp. 837–844.

[6] J. Richalet and D. O'Donovan. *Predictive Functional Control: Principles and Industrial Applications*. Springer, 2009.